

1. Develop a Python program to implement a **Naïve Bayes classifier** for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.

Q1: Naive Bayes Classifier

With and Without Laplace Smoothing

```
from collections import Counter
```

```
# Training dataset
```

```
# Features: Outlook, Wind
```

```
# Class: Play
```

```
data = [  
    ("Sunny", "Weak", "Yes"),  
    ("Sunny", "Strong", "No"),  
    ("Rainy", "Weak", "Yes"),  
    ("Rainy", "Strong", "No"),  
    ("Overcast", "Weak", "Yes"),  
    ("Overcast", "Strong", "Yes"),  
    ("Sunny", "Weak", "Yes"),  
    ("Rainy", "Weak", "Yes")  
]
```

```
test_data = [  
    ("Sunny", "Weak", "Yes"),  
    ("Sunny", "Strong", "No"),  
    ("Rainy", "Weak", "Yes"),  
    ("Rainy", "Strong", "No")  
]
```

```
def train_naive_bayes(data, smoothing=False):
```

```
    classes = set(row[2] for row in data)
```

```
    class_counts = Counter(row[2] for row in data)
```

```
    total = len(data)
```

```
    # Prior probability
```

```
    priors = {  
        c: class_counts[c] / total  
        for c in classes
```

```

}
# Conditional probabilities
conditional = {}

for c in classes:
    class_rows = [row for row in data if row[2] == c]

    for feature_index in range(2):
        feature_values = set(row[feature_index] for row in data)

        for value in feature_values:
            count = sum(
                1 for row in class_rows
                if row[feature_index] == value
            )

            if smoothing:
                probability = (count + 1) / (
                    len(class_rows) + len(feature_values)
                )
            else:
                probability = count / len(class_rows)

            conditional[(c, feature_index, value)] = probability

return priors, conditional

```

```

def predict(instance, priors, conditional, smoothing=False):
    classes = priors.keys()
    scores = {}

    for c in classes:
        probability = priors[c]

        for feature_index, value in enumerate(instance):
            p = conditional.get((c, feature_index, value), 0)

            # If probability is zero, classification becomes zero
            if p == 0:
                probability = 0
                break

```

```

        probability *= p
        scores[c] = probability

    return max(scores, key=scores.get)

def calculate_accuracy(test_data, priors, conditional):
    correct = 0

    for row in test_data:
        prediction = predict(row[:2], priors, conditional)

        if prediction == row[2]:
            correct += 1

    return (correct / len(test_data)) * 100

# Without Laplace smoothing
priors_no, conditional_no = train_naive_bayes(
    data, smoothing=False
)

# With Laplace smoothing
priors_laplace, conditional_laplace = train_naive_bayes(
    data, smoothing=True
)

print("NAIVE BAYES CLASSIFIER")
print("-----")

print("\nPrior Probabilities:")

for c in priors_no:
    print(c, "=", round(priors_no[c], 3))

print("\nPredictions WITHOUT Laplace Smoothing:")

for row in test_data:
    prediction = predict(

```

```
    row[:2],
    priors_no,
    conditional_no
)
```

```
print(
    "Input:",
    row[:2],
    "Actual:",
    row[2],
    "Predicted:",
    prediction
)
```

```
print("\nPredictions WITH Laplace Smoothing:")
```

```
for row in test_data:
    prediction = predict(
        row[:2],
        priors_laplace,
        conditional_laplace,
        smoothing=True
    )
```

```
print(
    "Input:",
    row[:2],
    "Actual:",
    row[2],
    "Predicted:",
    prediction
)
```

```
accuracy_no = calculate_accuracy(
    test_data,
    priors_no,
    conditional_no
)
```

```
accuracy_laplace = calculate_accuracy(
```

```

    test_data,
    priors_laplace,
    conditional_laplace
)
print("\nAccuracy WITHOUT Laplace Smoothing:",
      accuracy_no, "%")

print("Accuracy WITH Laplace Smoothing:",
      accuracy_laplace, "%")

```

OUTPUT:

NAIVE BAYES CLASSIFIER

Prior Probabilities:

No = 0.25

Yes = 0.75

Predictions WITHOUT Laplace Smoothing:

Input: ('Sunny', 'Weak') Actual: Yes Predicted: Yes

Input: ('Sunny', 'Strong') Actual: No Predicted: No

Input: ('Rainy', 'Weak') Actual: Yes Predicted: Yes

Input: ('Rainy', 'Strong') Actual: No Predicted: No

Predictions WITH Laplace Smoothing:

Input: ('Sunny', 'Weak') Actual: Yes Predicted: Yes

Input: ('Sunny', 'Strong') Actual: No Predicted: No

Input: ('Rainy', 'Weak') Actual: Yes Predicted: Yes

Input: ('Rainy', 'Strong') Actual: No Predicted: No

Accuracy WITHOUT Laplace Smoothing: 100.0 %

Accuracy WITH Laplace Smoothing: 100.0 %

2. Write a Python program to implement a simple Genetic Algorithm to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.

Q2: Simple Genetic Algorithm

```

# Maximize  $f(x) = x^2$ 

import random

# Parameters
POPULATION_SIZE = 6
CHROMOSOME_LENGTH = 5
GENERATIONS = 10
MUTATION_RATE = 0.1

# Convert binary chromosome to decimal
def chromosome_to_decimal(chromosome):
    binary = "".join(map(str, chromosome))
    return int(binary, 2)

# Fitness function
def fitness(chromosome):
    x = chromosome_to_decimal(chromosome)
    return x * x

# Generate random chromosome
def create_chromosome():
    return [
        random.randint(0, 1)
        for _ in range(CHROMOSOME_LENGTH) ]

# Create population
def create_population():
    return [
        create_chromosome()
        for _ in range(POPULATION_SIZE) ]

# Selection
def selection(population):
    population.sort(
        key=fitness,
        reverse=True
    )

# Select best two chromosomes

```

```
return population[:2]
```

```
# Crossover
```

```
def crossover(parent1, parent2):
```

```
    point = random.randint(
```

```
        1,
```

```
        CHROMOSOME_LENGTH - 1 )
```

```
    child1 = parent1[:point] + parent2[point:]
```

```
    child2 = parent2[:point] + parent1[point:]
```

```
    return child1, child2
```

```
# Mutation
```

```
def mutation(chromosome):
```

```
    for i in range(len(chromosome)):
```

```
        if random.random() < MUTATION_RATE:
```

```
            chromosome[i] = 1 - chromosome[i]
```

```
    return chromosome
```

```
# Main Genetic Algorithm
```

```
random.seed(10)
```

```
population = create_population()
```

```
print("GENETIC ALGORITHM")
```

```
print("-----")
```

```
for generation in range(GENERATIONS):
```

```
    best = max(
```

```
        population,
```

```
        key=fitness
```

```
    )
```

```
    best_x = chromosome_to_decimal(best)
```

```
    best_fitness = fitness(best)
```

```
    print(
```

```

"Generation",
generation + 1,
": Best x =",
best_x,
"Fitness =",
best_fitness
)

# Selection
parents = selection(population)

new_population = parents.copy()

# Create new population
while len(new_population) < POPULATION_SIZE:

    parent1 = random.choice(parents)
    parent2 = random.choice(parents)

    child1, child2 = crossover(
        parent1,
        parent2
    )

    child1 = mutation(child1)
    child2 = mutation(child2)

    new_population.append(child1)

    if len(new_population) < POPULATION_SIZE:
        new_population.append(child2)

    population = new_population

# Final best solution
best = max(
    population,
    key=fitness
)

best_x = chromosome_to_decimal(best)
best_fitness = fitness(best)

print("\nBest Solution")

```

```
print("-----")
print("Chromosome:", ".join(map(str, best)))
print("x =", best_x)
print("Maximum Fitness =", best_fitness)
```

OUTPUT:

GENETIC ALGORITHM

```
-----
Generation 1 : Best x = 30 Fitness = 900
Generation 2 : Best x = 30 Fitness = 900
Generation 3 : Best x = 30 Fitness = 900
Generation 4 : Best x = 30 Fitness = 900
Generation 5 : Best x = 30 Fitness = 900
Generation 6 : Best x = 30 Fitness = 900
Generation 7 : Best x = 30 Fitness = 900
Generation 8 : Best x = 30 Fitness = 900
Generation 9 : Best x = 30 Fitness = 900
Generation 10 : Best x = 30 Fitness = 900
```

Best Solution

```
-----
Chromosome: 11110
x = 30
Maximum Fitness = 900
```

3. Write a Python program to implement the Support Vector Machine (SVM) algorithm for binary classification. Use a simple dataset and implement a linear kernel. Train the model, classify test instances, and display accuracy. Analyze how the margin and support vectors influence classification performance.

```
# Q3: Support Vector Machine
# Linear SVM using Scikit-learn
```

```
from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# Simple binary classification dataset
```

```
X = [
    [1, 2],
    [2, 3],
    [2, 1],
```

```
[3, 2],
[6, 7],
[7, 8],
[8, 7],
[7, 6]
]
# Class labels
# 0 = Class A
# 1 = Class B

y = [
    0, 0, 0, 0,
    1, 1, 1, 1
]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)

# Create SVM model
model = svm.SVC(
    kernel='linear',
    C=1.0
)

# Train model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(
    y_test,
    y_pred
```

)

```
print("SUPPORT VECTOR MACHINE")  
print("-----")
```

```
print("\nTraining Data:")  
for x, y_value in zip(X_train, y_train):  
    print(x, "Class =", y_value)
```

```
print("\nTest Data:")  
for x, actual, predicted in zip(  
    X_test,  
    y_test,  
    y_pred  
):  
    print(  
        x,  
        "Actual =",  
        actual,  
        "Predicted =",  
        predicted  
    )
```

```
print("\nAccuracy:",  
      accuracy * 100,  
      "%")
```

```
# Display support vectors  
print("\nSupport Vectors:")
```

```
for vector in model.support_vectors_:  
    print(vector)
```

```
# Display number of support vectors  
print(  
    "\nNumber of Support Vectors:",  
    len(model.support_vectors_) )
```

```
# Display coefficients
print("\nLinear SVM Coefficients:")
print(model.coef_)

print("\nIntercept:")
print(model.intercept_)
```

OUTPUT:

SUPPORT VECTOR MACHINE

Training Data:

```
[1, 2] Class = 0
[7, 6] Class = 1
[2, 1] Class = 0
[6, 7] Class = 1
[3, 2] Class = 0
[8, 7] Class = 1
```

Test Data:

```
[2, 3] Actual = 0 Predicted = 0
[7, 8] Actual = 1 Predicted = 1
```

Accuracy: 100.0 %

Support Vectors:

```
[3. 2.]
[7. 6.]
```

Number of Support Vectors: 2

Linear SVM Coefficients:

```
[[0.25 0.25]]
```

Intercept:

```
[-2.25]
```